While I recommend using an external identity provider where possible, sometimes you really want to store all the authentication details of your users directly in your app. That's the approach I describe in this chapter.

ASP.NET Core Identity (sometimes shortened to Identity) is a system that simplifies building the user-management aspect of your app. It handles all the boilerplate for saving and loading users to a database, as well as best practices for security, such as user lockout, password hashing, and *multifactor authentication*.

DEFINITION *Multifactor authentication (MFA)*, and the subset *two-factor authentication* (2FA) require both a password and an extra piece of information to sign in. This could involve sending a code to a user's phone by Short Message Service (SMS) or using a mobile app to generate a code, for example.

In the next section I'm going to talk about the ASP.NET Core Identity system, the problems it solves, when you'd want to use it, and when you might not want to use it. In section 23.3 we take a look at some code and see ASP.NET Core Identity in action.

23.2 What is ASP.NET Core Identity?

Whenever you need to add nontrivial behaviors to your application, you typically need to add users and authentication. That means you'll need a way of persisting details about your users, such as their usernames and passwords.

This might seem like a relatively simple requirement, but given that this is related to security and people's personal details, it's important you get it right. As well as storing the claims for each user, it's important to store passwords using a strong hashing algorithm, to allow users to use MFA where possible, and to protect against brute-force attacks, to name a few of the many requirements. Although it's perfectly possible to write all the code to do this manually and to build your own authentication and membership system, I highly recommend you don't.